SSE与WebSocket技术对比调研报告

在现代Web应用架构中，实时数据交互已成为提升用户体验的核心要素。服务器发送事件（Server-Sent Events, SSE）与WebSocket是两种广泛采用的实时通信技术。尽管二者均旨在替代传统的HTTP短轮询与长轮询机制，但在底层协议设计、通信模型、性能开销及工程实践上存在本质差异。本报告基于现有技术资料，对二者进行系统性对比分析，为架构选型提供客观依据。

一、 核心架构与底层协议

1.1 协议基础与网络层级

SSE是建立在标准HTTP/1.1协议之上的轻量级推送技术[2]。它完全复用现有的HTTP基础设施，通过响应头`Content-Type: text/event-stream`标识数据流，无需引入独立的网络协议栈[1]。相比之下，WebSocket是一种独立于HTTP的全双工通信协议，于2011年由IETF标准化为RFC 6455[4]。WebSocket在初始阶段依赖HTTP的`Upgrade: websocket`请求完成握手协商，但连接建立后即脱离HTTP协议，转为基于TCP传输层的独立帧协议[1]。该协议默认复用HTTP的80与443端口，并原生支持TLS加密（`wss://`）[4]。

1.2 连接建立与维护机制

SSE的连接建立等同于一次标准的HTTP GET请求，服务器接收后保持连接处于打开状态，并持续向客户端推送事件流[2]。WebSocket则需经历专门的握手阶段，双方交换密钥并确认协议升级后，建立持久的TCP长连接[3]。在连接生命周期管理上，WebSocket会话结束后需显式执行关闭流程以释放底层资源[5]。SSE连接则高度依赖客户端的`EventSource` API，该组件在底层封装了连接状态管理，大幅简化了服务端维护成本[1]。

二、 通信模型与数据格式

2.1 通信方向与交互模式

通信方向是区分两者的核心指标。SSE采用严格的单向通信模型，数据流仅能从服务器单向推送至客户端，客户端无法通过该连接主动回传业务数据[1]。WebSocket提供全双工

（Full-Duplex）通信能力，客户端与服务器可在同一TCP连接上同时、独立地发送与接收消息，实现真正的双向实时交互[3]。这种架构差异直接决定了两者在复杂交互场景中的适用边界。

2.2 数据载荷与编码支持

在数据格式方面，SSE受限于HTTP文本流规范，仅支持UTF-8编码的纯文本数据传输，无法直接承载二进制文件[1]。WebSocket协议设计更为底层，原生支持文本（Text）与二进制（Binary）两种帧类型，可直接传输`Blob`、`ArrayBuffer`或自定义序列化数据，适用于对带宽效率要求极高的场景[1]。此外，WebSocket的消息以帧为单位进行封装，支持分片传输与压缩优化，进一步提升了大数据量传输的稳定性[3]。

三、 性能开销与工程特性

3.1 延迟表现与资源消耗

WebSocket在握手完成后，后续数据交换完全剥离了HTTP头部信息，显著降低了网络传输开销与往返延迟，具备极高的带宽利用率[5]。SSE虽然避免了频繁轮询，但每次推送的数据仍封装在HTTP响应流中，且基于HTTP长连接的特性使其在经过反向代理或负载均衡器时，可能面临连接超时中断的风险[3]。从服务器资源角度看，SSE对现有HTTP服务改造较小，并发压力相对可控；而WebSocket需要独立的连接池管理与状态维护，对服务器的内存与文件描述符消耗更高[3]。

3.2 自动重连与生态兼容性

SSE的`EventSource`接口内置了断线自动重连机制，浏览器会在网络异常时自动尝试恢复连接，开发者无需编写额外的重试逻辑[1]。WebSocket协议本身未定义自动重连规范，需依赖应用层实现心跳保活（Ping/Pong）与断线重连策略，或借助Socket.IO等第三方库提供HTTP降级兼容方案[5]。在浏览器支持方面，现代主流浏览器均原生支持两者，但SSE在旧版环境（如IE）中完全不可用，而WebSocket可通过Polyfill进行平滑降级[1]。

四、 综合对比与场景选型

4.1 核心差异对比表

对比维度	SSE (Server-Sent Events)	WebSocket
通信方向	单向（服务器→客户端）	双向全双工
底层协议	HTTP/1.1 长连接	独立TCP协议 (ws/wss)
数据格式	仅支持UTF-8文本	支持文本与二进制数据
连接管理	`EventSource` 自带自动重连	需手动实现心跳与重连逻辑
网络开销	较高（保留HTTP头部）	极低（无冗余头部）
服务器压力	较低，易集成至现有HTTP服务	较高，需专用连接池管理
典型场景	新闻推送、日志监控、状态通知	在线聊天、多人游戏、实时协作

4.2 选型建议

技术选型应严格遵循业务数据流向与性能指标。若应用场景仅需服务器向客户端单向推送实时数据（如股票行情、系统告警、AI流式响应），且追求开发效率与部署简便性，SSE是更优选择[2]。若应用涉及高频双向交互、毫秒级延迟要求或需传输二进制数据（如即时通讯、在线对战游戏、实时音视频信令），则必须采用WebSocket[1]。在复杂企业级架构中，可采用混合策略：使用SSE处理低频状态广播，利用WebSocket承载核心高频交互逻辑，以平衡性能与运维成本。

五、 结论

SSE与WebSocket并非技术替代关系，而是针对不同实时通信需求的互补方案。SSE凭借基于HTTP的轻量架构与原生自动重连特性，在单向数据推送场景中具备极高

[truncated]

参考文献

[1] cnblogs.com — WebSocket 与 SSE 对比：即时通信的选择（一） - 何双新 - 博客园 URL: https://www.cnblogs.com/1314520xh/p/18805931 原文片段：「WebSocket 与 SSE 对比：即时通信的选择（一） - 何双新 - 博客园 且将新火试新茶 WebSocket 与 SSE 对比：即时通信的选择（一） 在现代 Web 开发中，实时通信需求越来越多，比如聊天应用、实时通知、直播弹幕、股票行情推送等。实现这些需求的常见技术有 WebSocket 和 SSE（Server-Sent Events），但它们各有优缺点，适用于不同的场景。本文将详细介绍 WebSocket 和 SSE，并进行对比分析，帮助你选择合适的技术方案。 1. WebSocket 介绍 WebSocket 是 HTML5 引入的...」

[2] blog.csdn.net — SSE 和 Websocket 的比较_sse和websocket区别-CSDN博客 URL: https://blog.csdn.net/weixin_42560424/article/details/131694850 原文片段：「SSE 和 Websocket 的比较_sse和websocket区别-CSDN博客 SSE 和 Websocket 的比较 最新推荐文章于 2026-06-17 20:03:04 发布 原创 最新推荐文章于 2026-06-17 20:03:04 发布 · 9.5k 阅读 · 1 · 12 · 收录于 当前文章被以下社区和专栏收录： 本文详细解析了SSE和WebSocket两种实时通信技术。SSE是基于HTTP的单向通信机制，适用于实时更新数据；WebSocket是全双工通信协议，支持双向通信，适用于实时互动。文章从通信模型、连接性、实时性等方面比较了...」

[3] apifox.com — SSE 和 WebSocket 的区别，差异对比 URL: https://apifox.com/apiskills/sse-vs-websocket 原文片段：「SSE 和 WebSocket 的区别，差异对比 SSE 和 WebSocket 的区别，差异对比 SSE（Server-Sent Events）和 WebSocket 都是用于实现服务器与客户端之间实时双向通信的技术。虽然它们都可以用于实时更新数据，但它们在实现方式、特点和适用场景上有着明显的区别。 用 Apifox，节省研发团队的每一分钟 SSE 和 WebSocket 的区别，差异对比 免费使用 Apifox 相关推荐 最新文章 API 一体化协作平台 API 设计 API 文档 API 调试 自动化测试 API Mock API Hub 立即体验...」

[4] softwaremill.com — SSE vs WebSockets: Comparing Real-Time Communication Protocols URL: https://softwaremill.com/sse-vs-websockets-comparing-real-time-communication-protocols 原文片段：「SSE vs WebSockets: Comparing Real-Time Communication Protocols Contents Contents SSE vs WebSockets: Comparing Real-Time Communication Protocols SSE Websocket Communication Protocols Bartłomiej Żyliński 26 Feb 2024. 11 minutes read In today's text, I want to take a closer look at...」

[5] ably.com — ably.com URL: https://ably.com/blog/websockets-vs-sse 原文片段：「On this page WebSockets and Server-Sent Events are commonly used in realtime applications where quick and efficient data transfer is a critical requirement. The expectations of realtime experiences in applications has only grown with time, with improving technology and understand...」